

CPS 200

Lecture 04

Jan 21, 2026

Kyu Hur

Announcement

- Syllabus is on [Blackboard](#) (under General) and [CPS 200](#) website (under Supplementary)
- Grade weighting
 - a. Quizzes (30%)
 - b. Final Exam (30%)
 - c. Midterm (20%)
 - d. Group Project (20%)

Announcement

- **No part of your grade will be from attendance. Quizzes act as attendance; please make sure you pay attention in class and study!**
- Best way to study is to do & finish the exercises for each chapter
- If you do not do the exercises at home, you will not understand the material
- You need to review the materials (Lectures + Colab) and most importantly, write your own code and test out ideas
- If you have questions, come to office hours (Wed, 19:45-20:15) or email me

Announcement

- Quizzes
 - There are 16 total, you can miss 1
 - If you are absent, that will be your missed quiz (no makeups)
 - If you take all 16 quizzes, final quiz will be fully extra credit
 - All quizzes will have bonus questions for additional points
- **Midterm (Feb 9)**
- **Final (Apr 1) — Tentative**
- Try not to miss the midterm or final; makeups will be a slightly harder variation

Announcement

- Group Project is TBD
 - Will be with a partner
 - Will need to build a small system
 - AI use is discouraged
 - Will need to present your system in-class and I will ask questions on your project

From Lecture 03

Variables

- What rules are there for **variable** naming? Discuss with partners.
 - Can't start with a number
 - Can't contain any special characters e.g. @\$%^&*()#><,!?
 - The **underscore** or `_` is the only special character that is allowed to be part of a variable name
 - Can't use a **keyword** or **reserved keyword**

Variables

- Variable names are case sensitive (e.g. `Hello` and `HELLO` would be two different variables)

Keywords

- What is a **keyword** or **reserved keyword**? Discuss with partners.
- Keywords or reserved keywords are special word used to specify the structure of a program e.g.
 - `True, False, None, class, ...`
- There are 35 in total but it's also not good to name variables as functions e.g.
 - `int, str, len, etc.`

Functions

- What is a **function**? Can you write a function called `print_hello` that prints “Hello”? Can you **call** that function? Try it with your partners.

```
def print_hello():  
    print("Hello")
```

```
print_hello()
```

Functions

- Why is it `print_hello()`, and not `print_hello`? Discuss with your partners.
- How about **referencing a variable**? Why is that different?

```
def print_hello():  
    print("Hello")
```

```
print_hello()  
x = 0  
x
```

Functions

```
def print_hello():  
    print("Hello")
```

```
x = 0
```

```
x
```

```
print_hello
```

```
print_hello()
```

Python

Stack

print_hello

x

Functions

- Can you try writing a **function** (`function_2`) that **calls** another **function** (`function_1`)? `function_1` should print “Hello”. Try with your partners.

```
def function_1():  
    print("Hello")
```

```
def function_2():  
    function_1()
```

```
function_2()
```

Functions

- What is a **local variable**? Can you provide an example in a function?
Try writing a function called `repeat()` that takes two arguments
(`string`, `num`)
- Should `repeat string` the `num` number of times
- Store that repeated string in a **local variable** before printing

```
def repeat(string, num):  
    repeated_string = string * num  
    print(repeated_string)
```

```
string = "hello"  
repeat(string, 5)
```

Parameters and Arguments

- What is a **parameter**? An **argument**?
 - A **parameter** is the variable defined in a function's declaration
 - An **argument** is the actual value passed to that function when it is called

```
def function_1(parameter):  
    print(parameter)
```

```
argument = 4  
function_1(argument)
```

Lecture 04

Partial Imports

- Importing from a library was as simple as doing `import <library>`
- Importing the entire library is actually not good practice because it can cause namespace collisions e.g.
 - `something.pi` **VS** `math.pi`

Partial Imports

- We can partially import only what we need from a library with the keyword `from`, e.g.
 - `from math import pi`
 - `from math import sqrt`
 - `from math import pow`

Loops

- Discussed briefly last week, loops are extremely important in programming
- It “automates” repetitive tasks e.g. below
 - Both left and right have the same output

```
print(0)
print(1)
print(2)
print(3)
...
print(9)
```

```
for i in range(10):
    print(i)
```

Loops

- Dissecting the structure of a loop, we get the following:
 - **for** and **in**, which are **reserved keywords**
 - **range()**, which is a base Python function (to be discussed next slide)
 - **i**, which is short for iterator. This however, is a variable that YOU name so it can be anything (e.g. `x`, `y`, `num`, `number`, `string`, etc)

```
for i in range(10) :  
    print(i)
```

Loops

```
for value in range(10):  
    print(value)
```

```
for num in range(10):  
    print(num)
```

```
for x in range(10):  
    print(x)
```

All valid!!

Loops

- The `range()` function is very useful to run loops
- It takes 1 argument, with 2 additional optional arguments
 - `range(stop)`
 - Ranges from 0 to `stop`
 - `range(start, stop)`
 - Ranges from `start` to `stop`
 - `range(start, stop, step)`
 - Ranges from `start` to `stop` in `step`

`range(stop)`

```
for i in range(4):  
    print(i)
```

- Ranges from 0 (default) to the number passed (4 in this case)
- 0 is inclusive while the `stop` value is exclusive
- What will the statement above print? Discuss with partners

`range(start, stop)`

```
for i in range(1, 4):  
    print(i)
```

- Ranges from 1st argument to the 2nd argument
- 1st argument is inclusive and 2nd argument is exclusive
- What will the statement above print? Discuss with partners

`range(start, stop, step)`

```
for i in range(1, 4, 1):  
    print(i)
```

- Same as before, but the 3rd argument is the **step** (e.g. if it's 2, then it will increase in steps of 2)
- What will the statement above print? Discuss with partners

```
range(start, stop, step)
```

```
for i in range(1, 4, 2):  
    print(i)
```

- What will the statement above print? Discuss with partners

```
range(start, stop, step)
```

```
x = "hello"
```

```
y = len(x)
```

```
for i in range(y):
```

```
    print(i)
```

- What will the statement above print? Discuss with partners

Loops

```
for i in [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]:  
    print(i)
```

```
for i in range(10):  
    print(i)
```

These two blocks of code are functionally the same.

- Essentially, a `for` loop is iterating over a list of values e.g. above
- The `range()` function essentially creates this list of values on the fly based on the arguments provided
 - We will learn more about lists later in Chapter 9 (Lecture 07)

jupyterturtle

- Today, we will use the jupyterturtle library to draw diagrams on a canvas using a “turtle”
 - How this library was built is not important
 - You just need to learn the rules to manipulate the turtle

jupyterturtle

- You can “create” an instance of the turtle with `make_turtle()`
- `forward()` moves the turtle forward
 - Takes an argument that determines how far it travels
- `left()` and `right()` turns the face of the turtle left or right
 - Each takes one argument that turns it X number of degrees

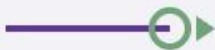
jupyterturtle

```
make_turtle()
```



jupyterturtle

```
make_turtle()  
forward(50)
```



jupyterturtle

```
make_turtle()  
forward(50)  
left(90)
```



jupyterturtle

```
make_turtle()  
forward(50)  
left(90)  
forward(50)
```



Booleans

- **Booleans** are another primitive **type** like int, str, float
 - In Python, the function for booleans are **bool**
 - A boolean is a variable that has one of only two values:
 - True
 - False
 - They are very important for **conditionals** (often called if statements)
- Booleans show up a lot for **conditional** statements

Booleans

- Some examples of **boolean** statements are shown below
- Can you guess what the output of each will be if $x = 2$ and $y = 3$?
 - $x < y$
 - $x > y$
 - $x == y$
 - $x != y$
 - $x >= y$
 - $x <= y$

Booleans

- What will the following code output?
- What is the data **type** for z?

```
x = 2
y = 3
z = (x < y)
print(z)
```

Conditionals

- In the heart of programming, there are **if** statements
 - If a statement is true, the code is executed
 - If a statement is false, the code is skipped
- What does the following code print? Discuss with partners.

```
x = random.randint(-1, 1)
if x > 0:
    print("x bigger than 0")
```

Note: The part highlighted in yellow is evaluated as a **boolean** (True / False)

Conditionals

- The **else** clause executes some other code if the “if part” is not true
- What does the following code print? Discuss with partners.

```
x = random.randint(-1, 1)
if x > 0:
    print("x bigger than 0")
else:
    print("x smaller or equal to 0")
```

Conditionals

- Python also has **elif** which is a chained conditional
- There can be any number of **elif** clauses

```
x = random.randint(-1, 1)
if x > 0:
    print("x bigger than 0")
elif x < 0:
    print("x smaller than 0")
else:
    print("x equal to 0")
```


Think Python

Think Python: Functions and Interfaces

- [Chapter 04.ipynb](#)

End